In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# 1. Load survey file
# ----------------------------

file_path = "Форма промежуточной оценки курса (draft) (Responses) - Form Res

survey = pd.read_csv(file_path)

# ----------------------------
# 2. Rename columns into English
# ----------------------------

survey = survey.rename(columns={
    "Как ты оцениваешь курс в целом прямо сейчас?": "overall_rating",
    "Оправдал ли курс твои ожидания?": "expectations",
    "Насколько вероятно, что ты порекомендуешь этот курс Илмхоны другу?": "r
    "Как ты оцениваешь качество объяснений преподавателя? ": "instructor_qua
    "Темп обучения для тебя: ": "course_pace",
    "Что в учебных материалах работает лучше всего? ": "best_materials",
    "Что тебе мешает учиться или замедляет прогресс?": "learning_barriers",
    "Насколько ты чувствуешь себя комфортно задавать вопросы инструктору?":
    "Как атмосфера в группе влияет на твою мотивацию? ": "group_motivation",
    "Ты уже применяешь навыки курса вне занятий? ": "outside_application",
    "Если бы ты мог(ла) изменить одну вещь в программе курса прямо сейчас —
    "Что тебя больше всего мотивирует продолжать обучение? ": "main_motivati
    "Насколько уверенно ты чувствуешь себя в материале курса по сравнению с
    "Планируешь ли ты продолжать изучать этот курс?": "continue_learning",
    "Что команда Илмхоны может сделать прямо сейчас, чтобы сделать твое обуч
})

# ----------------------------
# 3. Convert numeric columns
# ----------------------------

numeric_cols = [
    "overall_rating",
    "recommend_score",
    "instructor_quality",
    "question_comfort",
    "group_motivation",
    "confidence_growth"
]

for col in numeric_cols:
    survey[col] = pd.to_numeric(survey[col], errors="coerce")

# ----------------------------
# 4. Basic KPI summary
# ----------------------------
```

```
kpi_summary = pd.DataFrame({
    "Metric": [
        "Responses",
        "Avg overall course rating",
        "Avg recommendation score",
        "Avg instructor quality",
        "Avg comfort asking questions",
        "Avg group motivation",
        "Avg confidence growth"
    ],
    "Value": [
        len(survey),
        survey["overall_rating"].mean(),
        survey["recommend_score"].mean(),
        survey["instructor_quality"].mean(),
        survey["question_comfort"].mean(),
        survey["group_motivation"].mean(),
        survey["confidence_growth"].mean()
    ]
})

print("\nSurvey KPI Summary:")
print(kpi_summary.to_string(index=False))

# ----------------------------
# 5. Correlation analysis
# ----------------------------
# This helps show what seems most connected to satisfaction/recommendation/c
# Small sample size, so treat this as exploratory.

drivers = [
    "instructor_quality",
    "question_comfort",
    "group_motivation",
    "confidence_growth"
]

outcomes = [
    "overall_rating",
    "recommend_score",
    "confidence_growth"
]

correlations = []

for outcome in outcomes:
    for driver in drivers:
        if outcome != driver:
            correlations.append({
                "outcome": outcome,
                "driver": driver,
                "correlation": survey[[outcome, driver]].corr().iloc[0, 1]
            })

correlation_df = pd.DataFrame(correlations)
correlation_df = correlation_df.sort_values("correlation", ascending=False)
```

```
print("\nStudent Success Driver Correlations:")
print(correlation_df.to_string(index=False))

# ----------------------------
# 6. Category summaries
# ----------------------------

def category_summary(column_name):
    return (
        survey[column_name]
        .value_counts(dropna=False)
        .reset_index()
        .rename(columns={
            "index": column_name,
            column_name: "count"
        })
    )

expectations_summary = category_summary("expectations")
pace_summary = category_summary("course_pace")
outside_application_summary = category_summary("outside_application")
motivation_summary = category_summary("main_motivation")
continue_summary = category_summary("continue_learning")

print("\nExpectations Summary:")
print(expectations_summary.to_string(index=False))

print("\nCourse Pace Summary:")
print(pace_summary.to_string(index=False))

print("\nOutside Application Summary:")
print(outside_application_summary.to_string(index=False))

print("\nMain Motivation Summary:")
print(motivation_summary.to_string(index=False))

print("\nContinue Learning Summary:")
print(continue_summary.to_string(index=False))

# ----------------------------
# 7. Simple text theme tagging
# ----------------------------
# This turns open-ended responses into rough themes.

def tag_themes(text):
    text = str(text).lower()

    themes = []

    if any(word in text for word in ["практи", "practice", "задани", "homewo
        themes.append("More practice / assignments")
    if any(word in text for word in ["пример", "case", "real", "реаль"]):
        themes.append("More real-world examples")
    if any(word in text for word in ["повтор", "медлен", "slow"]):
        themes.append("Less repetition / faster pace")
```

```
    if any(word in text for word in ["обрат", "feedback", "ошиб", "разбор"]):
        themes.append("More feedback")
    if any(word in text for word in ["ничего", "не чего", "всё", "все", "нор
        themes.append("No major issue")

    if not themes:
        themes.append("Other")

    return themes

theme_rows = []

for col in ["learning_barriers", "one_change", "team_improvement"]:
    for response in survey[col].dropna():
        for theme in tag_themes(response):
            theme_rows.append({
                "question": col,
                "theme": theme
            })

theme_df = pd.DataFrame(theme_rows)

theme_summary = (
    theme_df
    .groupby(["question", "theme"])
    .size()
    .reset_index(name="count")
    .sort_values(["question", "count"], ascending=[True, False])
)

print("\nOpen-Ended Theme Summary:")
print(theme_summary.to_string(index=False))

# ----------------------------
# 8. Student Experience Index
# ----------------------------
# A simple combined score for each response.

survey["student_experience_index"] = survey[[
    "overall_rating",
    "recommend_score",
    "instructor_quality",
    "question_comfort",
    "group_motivation",
    "confidence_growth"
]].mean(axis=1)

print("\nStudent Experience Index:")
print(survey["student_experience_index"].describe())

# ----------------------------
# 9. Save outputs
# ----------------------------

survey.to_csv("midcourse_survey_cleaned.csv", index=False)
kpi_summary.to_csv("survey_kpi_summary.csv", index=False)
```

```
correlation_df.to_csv("student_success_driver_correlations.csv", index=False
expectations_summary.to_csv("survey_expectations_summary.csv", index=False)
pace_summary.to_csv("survey_pace_summary.csv", index=False)
outside_application_summary.to_csv("survey_outside_application_summary.csv",
motivation_summary.to_csv("survey_motivation_summary.csv", index=False)
continue_summary.to_csv("survey_continue_learning_summary.csv", index=False)
theme_summary.to_csv("survey_open_ended_theme_summary.csv", index=False)

# ----------------------------
# 10. Charts
# ----------------------------

# KPI bar chart
avg_scores = pd.DataFrame({
    "metric": [
        "Overall rating",
        "Recommend score",
        "Instructor quality",
        "Question comfort",
        "Group motivation",
        "Confidence growth"
    ],
    "score": [
        survey["overall_rating"].mean(),
        survey["recommend_score"].mean(),
        survey["instructor_quality"].mean(),
        survey["question_comfort"].mean(),
        survey["group_motivation"].mean(),
        survey["confidence_growth"].mean()
    ]
})

plt.figure(figsize=(10, 6))
plt.barh(avg_scores["metric"], avg_scores["score"])
plt.xlabel("Average Score")
plt.title("Average Mid-Course Survey Scores")
plt.xlim(0, 10)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig("average_midcourse_survey_scores.png")
plt.show()

# Correlation chart
top_corr = correlation_df.dropna().sort_values("correlation", ascending=True

plt.figure(figsize=(10, 6))
plt.barh(
    top_corr["driver"] + " → " + top_corr["outcome"],
    top_corr["correlation"]
)
plt.xlabel("Correlation")
plt.title("Student Success Drivers: Correlation with Key Outcomes")
plt.tight_layout()
plt.savefig("student_success_driver_correlations.png")
plt.show()
```

```
# Pace chart
plt.figure(figsize=(8, 5))
plt.bar(pace_summary["course_pace"], pace_summary["count"])
plt.xlabel("Course Pace")
plt.ylabel("Responses")
plt.title("Student Feedback on Course Pace")
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.savefig("course_pace_feedback.png")
plt.show()

# Motivation chart
plt.figure(figsize=(9, 5))
plt.barh(motivation_summary["main_motivation"], motivation_summary["count"])
plt.xlabel("Responses")
plt.title("Main Motivation for Continuing the Course")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig("student_motivation_summary.png")
plt.show()

# Theme chart
theme_totals = (
    theme_df
    .groupby("theme")
    .size()
    .reset_index(name="count")
    .sort_values("count", ascending=True)
)

plt.figure(figsize=(9, 5))
plt.barh(theme_totals["theme"], theme_totals["count"])
plt.xlabel("Mentions")
plt.title("Themes from Open-Ended Student Feedback")
plt.tight_layout()
plt.savefig("student_feedback_themes.png")
plt.show()

# ----------------------------
# 11. Auto-generated takeaways
# ----------------------------

print("\nKey Takeaways:")

print(f"- Students rated the course {survey['overall_rating'].mean():.1f}/10
print(f"- Instructor quality scored {survey['instructor_quality'].mean():.1f
print(f"- Recommendation likelihood scored {survey['recommend_score'].mean()
print(f"- Confidence growth scored {survey['confidence_growth'].mean():.1f}/

best_driver = correlation_df.dropna().sort_values("correlation", ascending=F

print(
    f"- The strongest observed relationship was between {best_driver['driver
    f"and {best_driver['outcome']} with correlation {best_driver['correlatic
)
```

```
print("- Because the sample size is small, these should be treated as early

print("\nFiles saved:")
print("midcourse_survey_cleaned.csv")
print("survey_kpi_summary.csv")
print("student_success_driver_correlations.csv")
print("survey_open_ended_theme_summary.csv")
print("average_midcourse_survey_scores.png")
print("student_success_driver_correlations.png")
print("course_pace_feedback.png")
print("student_motivation_summary.png")
print("student_feedback_themes.png")
```

Survey KPI Summary:

Metric	Value
Responses	15.000000
Avg overall course rating	8.200000
Avg recommendation score	7.800000
Avg instructor quality	8.400000
Avg comfort asking questions	8.066667
Avg group motivation	7.200000
Avg confidence growth	7.533333

Student Success Driver Correlations:

outcome	driver	correlation
overall_rating	confidence_growth	0.814015
overall_rating	instructor_quality	0.771938
recommend_score	confidence_growth	0.701116
overall_rating	question_comfort	0.659582
recommend_score	instructor_quality	0.624333
confidence_growth	question_comfort	0.608204
confidence_growth	instructor_quality	0.577973
overall_rating	group_motivation	0.455216
recommend_score	question_comfort	0.330070
recommend_score	group_motivation	0.293070
confidence_growth	group_motivation	0.292279

Expectations Summary:

count	count
Соответствует ожиданиям	11
Частично соответствует	4

Course Pace Summary:

count	count
В самый раз	11
Немного медленный	2
Довольно быстрый - не успеваю усваивать	2

Outside Application Summary:

count	count
Пока только учебные задания	7
Да, работаю над своим проектом	5
Ещё не применяю	2
Да, пробую на фрилансе	1

Main Motivation Summary:

count	count
Личные цели / проект	7
Желание зарабатывать больше	3
Перспектива трудоустройства	3
Интерес к технологиям	2

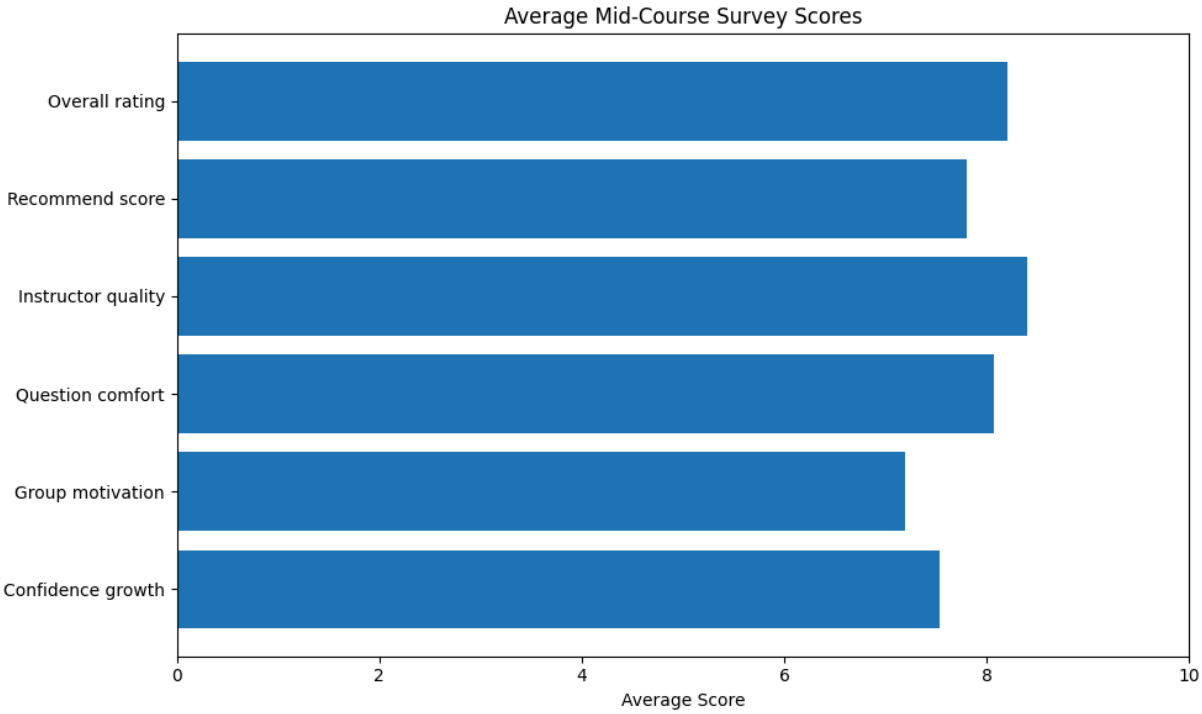
Continue Learning Summary:

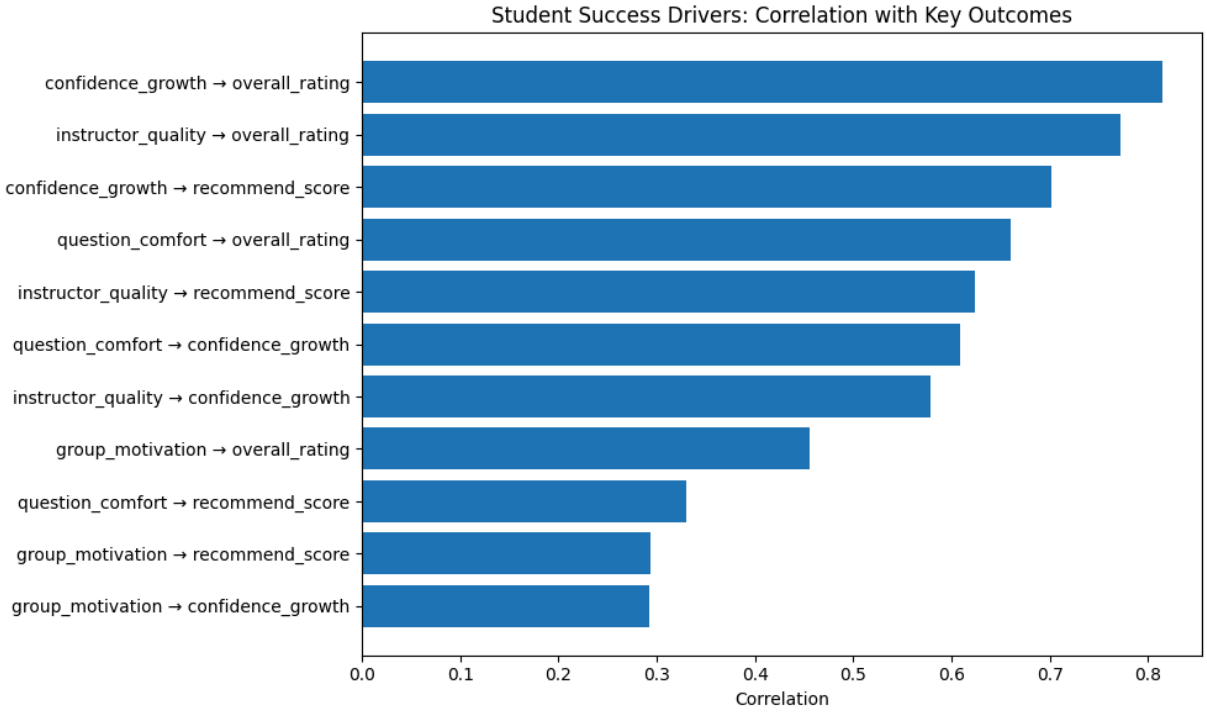
count	count
Да	13
Нет	1
Возможно	1

Open-Ended Theme Summary:

```
         question                          theme  count
learning_barriers                          Other      8
learning_barriers                  No major issue      5
learning_barriers   More practice / assignments      2
       one_change                          Other      8
       one_change                  No major issue      4
       one_change   More practice / assignments      3
       one_change Less repetition / faster pace      1
       one_change                   More feedback      1
 team_improvement                  No major issue      7
 team_improvement                          Other      5
 team_improvement   More practice / assignments      3

Student Experience Index:
count    15.000000
mean      7.866667
std       1.753907
min       2.833333
25%       7.583333
50%       7.833333
75%       9.000000
max      10.000000
Name: student_experience_index, dtype: float64
```




Student Success Drivers: Correlation with Key Outcomes
confidence_growth → overall_rating
instructor_quality → overall_rating
confidence_growth → recommend_score
question_comfort → overall_rating
instructor_quality → recommend_score
question_comfort → confidence_growth
instructor_quality → confidence_growth
group_motivation → overall_rating
question_comfort → recommend_score
group_motivation → recommend_score
group_motivation → confidence_growth
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Correlation

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-
packages/pandas/core/indexes/base.py:3812, in Index.get_loc(self, key)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:191, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:228, in pandas._libs.index.IndexEngine._get_loc_
duplicates()

KeyError: 'course_pace'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[3], line 281
    279 # Pace chart
    280 plt.figure(figsize=(8, 5))
--> 281 plt.bar(pace_summary[             ], pace_summary["count"])
    282 plt.xlabel("Course Pace")
    283 plt.ylabel("Responses")

File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-
packages/pandas/core/frame.py:4113, in DataFrame.__getitem__(self, key)
   4111 if self.columns.nlevels > 1:
   4112     return self._getitem_multilevel(key)
-> 4113 indexer = self.columns.get_loc(key)
   4114 if is_integer(indexer):
   4115     indexer = [indexer]

File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-
packages/pandas/core/indexes/base.py:3819, in Index.get_loc(self, key)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: 'course_pace'
```


In [ ]: